A Byte-Level Multilingual News Recommender for Microcontrollers

Abstract. Running a news recommender on the device itself keeps a reader's clicks private, but the usual models store a word-embedding table of tens to hundreds of megabytes per language, which no microcontroller's flash can hold. We avoid the table by reading a title as its raw UTF-8 bytes: a small char-CNN whose 256-value vocabulary is shared by every language. Around this encoder we search three families of architecture (unconstrained, budget-constrained and binary-aware) and evaluate each at full precision and with inner-layer weights at eight bits and at one bit, measuring ranking quality on MIND against estimated flash, RAM and energy. With eight-bit weights the constrained model scores 0.614 AUC at an estimated 204 KB and the unconstrained one 0.639 at 790 KB (means over three seeds). For reference, NRMS with GloVe vectors scores 0.664 over three seeds, a reduced NRMS with a random 256-dimensional table 0.604, and a per-user category histogram with no network 0.598. At the constrained architecture, starting from weights distilled from a frozen multilingual sentence-transformer adds 0.011 AUC (0.629 against 0.618), and training on clicks shown in a random xMIND language raises the mean over fourteen translations from 0.557 to 0.589 at a cost of 0.020 in English. Naive one-bit weights cost the constrained model 0.065 AUC (0.554); ReActNet blocks started from distilled weights reach 0.587 at 194 KB. Standard deviations over seeds reach 0.017, so smaller gaps are not resolved. Exported as an 8-bit file, the distilled constrained encoder scores 0.622 (0.628 at full precision) and runs in 31.2 ms per title on an STM32H7B3I-DK and 450 ms on a NUCLEO-F401RE. A μNAS search under the budgets of the two boards, with the final choice made after full training, finds encoders of 0.605 and 0.608 AUC against 0.599 and 0.589 for the best hand-designed shapes of the same budgets; the one for the NUCLEO-F401RE takes 60.6 ms per title there, against 88.7 ms for that shape. On boards of ours the two final choices draw about 7.3 mJ (F401) and 3.5 mJ (H7) per title from the USB port, and the H7 times of the same files vary by up to 2.4 times with the code generator's buffer layout. On an Android phone the 8-bit encoder takes 0.53 ms and about 0.95 mJ per title. Code and artefacts are released.

Keywords: Edge AI · Neural Architecture Search · Binarized Neural Networks · Multilingual News Recommendation · TinyML · Knowledge Distillation

1 Introduction

Running personalisation on the phone or sensor node rather than the cloud is attractive for privacy: the record of what a user reads never leaves the device.

Two difficulties come with that setting. A model that starts from nothing for each new user recommends poorly until it has seen enough clicks, and the standard neural recommenders (NRMS [23], NAML [22] and their relatives) represent a title with a word-embedding matrix. That matrix is large: GloVe-300d over 30–100k words is already 37–120 MB, one table per language, and none of it fits a microcontroller's few megabytes of flash.

We study both problems offline, on MIND [25] and its translated extension xMIND [7], and then shrink the result for the device. The idea is to drop words and read a title as its UTF-8 bytes. A byte takes 256 values, so the vocabulary is a single 256-row table shared by every language, and one small convolutional encoder reads Chinese, Georgian and English alike. A frozen multilingual teacher [19] supplies distillation targets that place a translated title near its English original; we use them to score candidate architectures and to initialise the encoder.

We contribute three things. A byte-level content encoder removes the embedding-table memory wall: its vocabulary is one 256-row table for every language. Around it we benchmark three architecture searches (unconstrained, budget-constrained and binary-aware) at three weight precisions, reporting accuracy alongside estimated flash, RAM and energy, and search again with μNAS under the budgets of two STM32 boards, measuring the results on the boards. Finally we measure what one-bit weights cost at this scale and how much of it ReActNet activations and Bi-Real shortcuts, started from distilled weights, recover. Character-level inputs have been used for click prediction [4], and vocabulary-free projections give text classifiers of about 200 KB [8]. A search of arXiv and IEEE Xplore (October 2026) returned no news recommender that reads raw bytes and none evaluated on a microcontroller; the nearest on-device NAS and binary work is on vision [11, 12, 17].

2 Related Work

Neural news recommendation on MIND is well charted. NRMS [23], NAML [22], LSTUR, NPA and DKN encode titles and users with attention or knowledge graphs; their published test AUC on the large split runs from 0.65 (DKN) to 0.68 (NRMS) [25]. Language-model recommenders (PLM-NR, Fastformer, UNBERT) reach 0.70–0.73 [24] but wrap a BERT-sized encoder of hundreds of megabytes, out of reach on a microcontroller; we keep NRMS as the reference that is closest in size. The tools we borrow are vocabulary-free character and byte models [26, 5, 3], cross-lingual distillation [19], memory-budgeted NAS [11], and the binary-network line of XNOR-Net [18], Bi-Real [14] and ReActNet [13]; what is new is putting them together for on-device multilingual news.

3 Method

We treat recommendation as impression-level ranking: score candidate titles against the titles a user has clicked, embedding both with the same encoder

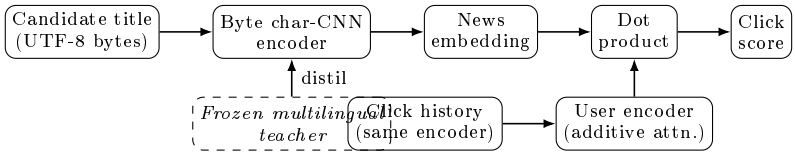


Fig. 1. The recommender: a byte char-CNN embeds candidate and history titles; an additive-attention encoder pools the history and a dot product scores the candidate. A frozen multilingual teacher provides distillation targets. Only the char-CNN is searched and quantised.

(Fig. 1). The encoder reads a title's bytes through a 256-entry embedding, a few depthwise-separable one-dimensional convolutions, a masked average over positions, and a linear projection. An additive-attention encoder summarises the user's history, and the score is the dot product of the two vectors, trained with one positive and four negatives redrawn at every step (AdamW). A frozen `paraphrase-multilingual-MiniLM-L12-v2` [21] supplies a 384-dimensional target for each English title, and because the xMIND titles are translations of those same articles the distillation loss maps every translation to that one target; a Matryoshka [9] term keeps the leading coordinates informative on their own. Distillation has two uses here: it is the fitness of the architecture search, and it initialises the encoder in the ablation and in the one-bit model of Sect. 5. The nine models of Table 1 are trained on clicks from a random initialisation.

The search has three arms: one unconstrained, one held to an on-device budget with eight-bit weights, and one held to the same budget with one-bit weights. The space holds 120 architectures (six channel counts, five depths, four output widths). Each arm runs an evolutionary search over it, scores a candidate by a short distillation, and rejects candidates over budget; precision is then the second axis. Quantisation is simulated on the weights with a straight-through estimator, per-channel for eight bits and sign-times-scale for one. Following XNOR-Net, the byte embedding and the outermost layers stay in full precision, and so do all activations, so the quantised rows cover 66%, 31% and 24% of the parameters of the three architectures. For the binary arm alone we add the ReActNet activations and Bi-Real shortcuts, binarise the block inputs and start from the distilled weights, which is what makes one-bit training converge to something useful. Sizes are parameter counts at each layer's precision and multiply-accumulates are counted per layer. The energy column multiplies them by per-operation figures: 4.6 and 0.23 pJ for a 32-bit float and an eight-bit integer multiply-accumulate at 45 nm [6], and one sixty-fourth of the former for a one-bit operation, the operation-count convention of binary networks [18, 14]. We read it only as a relative ordering, since it ignores the memory traffic that often dominates on a real chip.

For the boards we searched again, with the lab's one-dimensional version [2] of μNAS [10], because the June space holds only channel-depth grids and its operation count was wrong. The μNAS space is much wider: up to ten convolution blocks of standard, pointwise or depthwise layers (kernel 3 to 7, stride 1 or 2, op-

tional max-pooling, batch normalisation and ReLU), optional residual branches, and a global-average-pooling head with a 384-unit output. A candidate reads a title's bytes through the fixed byte table of the distilled reference encoder plus a padding-mask channel, and is distilled to the teacher on 96,000 rows in fifteen languages for at most fifteen epochs; its error is one minus the mean validation cosine. Aging evolution (150 candidates, population 40, sample 12) trades the error against the multiply-accumulates, eight-bit weight size and peak activation memory of the fork's resource model, under one budget per board: 256 KiB of weights, 128 KiB of activations and 2 M multiply-accumulates per title for the STM32H7B3I-DK, and 128 KiB, 48 KiB and 1 M for the NUCLEO-F401RE. The eight best candidates within budget are trained again with three seeds under the search recipe (the recipe choice). Because that short recipe ranks candidates differently from full training (Sect. 5), the final choice among the same eight is made after full distillation, by the mean validation cosine over three seeds. Both choices use validation rows only. The 30 hand-designed shapes of the June space with a 384-wide output are trained under the same recipe and costed by the same model. For the ranking experiments the chosen networks, the best hand-designed shape within each budget and 64-5-384 are rebuilt in PyTorch with this input and pooling (outputs within 2.3×10^{-5} of the Keras build) and trained like the reference encoder: distillation on all 769k training rows, then English clicks.

4 Experimental Setup

We use MINDsmall [25]: fifty thousand users, about 157k training and 73k development impressions over 51k articles, together with the fourteen NLLB translations of xMIND [7, 16]. xMIND provides only translated titles, keyed to the same article ids, so we reuse the English behaviour logs and swap the text. Every recommender we train, and the NRMS baseline, sees only English clicks, and the other languages appear only at evaluation. The multilingual numbers are therefore a test of cross-lingual transfer in the content encoder, not of native-language personalisation. We stay on the small split rather than the million-user MIND-large because the nine cells of Table 1 alone take 5.6 hours of training for three seeds on one laptop; the small split has no public test set, so we evaluate on dev with the number of epochs fixed in advance. The search runs a population of thirty over fifteen generations, scoring each candidate by a two-epoch distillation on twenty thousand titles. The budgets of the constrained arms are 256 KB of flash and 128 KB of RAM, about eight times tighter than an STM32H7 holds. A bound of 2 M multiply-accumulates was also set, but the counter of that run skipped the quantised layers, so both constrained winners (3.29 M and 3.26 M) exceed it. The μNAS searches of Sect. 3 enforce it with the corrected count. Everything runs on one laptop: an Intel Core Ultra 9 275HX with an eight-gigabyte RTX 5070 Laptop GPU and 32 GB of memory, PyTorch 2.12 on CUDA 13. The targets are STM32 microcontrollers and an Android phone. Board latency, flash and RAM come from the ST Edge AI Developer Cloud (ST Edge AI Core

Table 1. Architecture × weight precision on MINDsmall dev, mean over 3 seeds (AUC: mean ± standard deviation). Each architecture is trained from a random initialisation on English clicks; INT8 and binary are the trained model with simulated weight quantisation of the inner layers and two more epochs. **Bold**: best AUC per arm; †: precision the arm was searched at. Size and energy are estimates (Sect. 3).

Arm	Prec.	AUC	MRR	nDCG@10	Size (KB)	Energy (μJ)
NAS	FP32†	0.628±0.015	0.326	0.373	1566.8	166.3
NAS	INT8	**0.639±0.005**	0.333	0.381	789.8	17.91
NAS	Binary	0.598±0.014	0.302	0.350	563.1	12.54
Micro-NAS	FP32	**0.619±0.001**	0.327	0.371	266.8	15.15
Micro-NAS	INT8†	0.614±0.006	0.323	0.365	203.9	3.16
Micro-NAS	Binary	0.554±0.017	0.277	0.320	185.6	2.72
Bin. μNAS	FP32	**0.602±0.003**	0.301	0.350	311.4	14.98
Bin. μNAS	INT8	0.601±0.009	0.302	0.351	255.7	4.35
Bin. μNAS	Binary†	0.558±0.007	0.271	0.317	239.5	3.96

4.0.1) on an STM32H7B3I-DK (Cortex-M7, 280 MHz) and a NUCLEO-F401RE (Cortex-M4, 84 MHz). On a NUCLEO-F401RE and an STM32H7B3I-DK of ours, at the same clocks, we flashed ST's validation firmware (ST Edge AI Core 4.0.0) and a firmware that alternates the encoder in a loop with the MCU in STOP mode; a USB power meter (Joy-IT JT-UM120) between the laptop and the board reads both phases. Phone latency and energy come from a DOOGEE S98Pro (MediaTek Helio G96, two Cortex-A76 and six Cortex-A55 cores, Android 12), with ONNX Runtime 1.27 inside our reader app; the energy is read from the phone's own battery gauge, unplugged, over 15 minutes with the encoder idle and 15 minutes with it running. Every run is seeded (seeds 42, 1 and 2 for Table 1; 42, 12 and 1 for Tables 2 and 3), uses the loop of Sect. 3, which redraws the negatives at every step, and is reported as the mean and standard deviation over its three seeds. The June architectures are kept; the June run that found them drew its negatives once and did not seed the network initialisation. The downloaded data is pinned by checksum, and the code, notebook, result files and a dependency lock are released.[1]

5 Results

Table 1 collects the nine combinations (the three arms found the architectures 256-4-384, 64-5-384 and 96-2-384, with 36.1, 3.29 and 3.26 million multiply-accumulates per title). Two patterns emerge. Moving the inner weights from full to eight-bit precision changes AUC by at most 0.011, inside the spread over seeds, while the energy estimate falls 3.4 to 9.3 times and the size 1.2 to 2 times. One bit is different: it loses 0.029 to 0.065 AUC, and because the embedding and outer layers stay in full precision its size advantage over eight bits is small at this scale. The RAM estimate, the input and output buffer of the widest layer at one byte per activation, is 16 KB for the constrained model and 64 KB for the unconstrained one. On the laptop a title takes 0.10–0.34 ms

[1] `https://github.com/SepehrMohammady/MIND-Edge-Recommender`

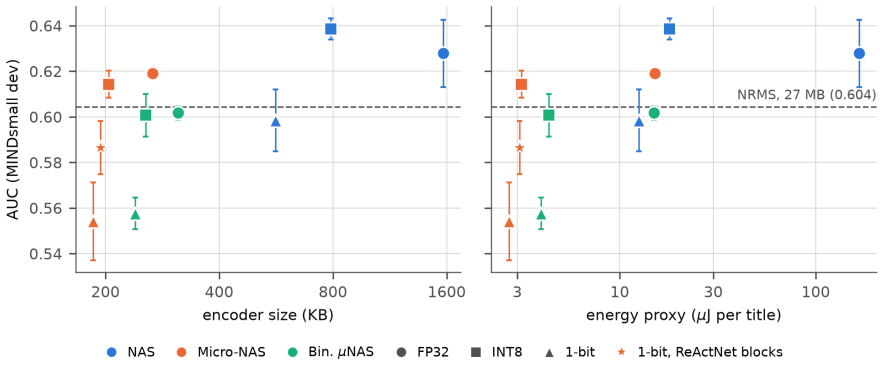


Fig. 2. Accuracy against estimated encoder size (left) and against the energy estimate (right), per arm and weight precision, mean and standard deviation over three seeds; star: one-bit model with ReActNet blocks started from distilled weights (three seeds); dashed line: reduced NRMS with a 27 MB word table (three seeds).

on CPU (onnxruntime) and under 0.7 ms on GPU for the three full-precision architectures.

Table 2 places these numbers. Our NRMS reproduction in the reduced configuration of the June run (a randomly initialised 256-dimensional word table; 92% of its 7.08 million parameters are the table, 27 MB) averages 0.604 over three seeds with a spread of 0.010, which covers the 0.607 of the June run. In the reference configuration shipped with MIND (GloVe-initialised 300-dimensional table, twenty heads) NRMS reaches 0.664, in line with the 0.656 reported for this split [15]. Two scorers with no network set a floor: popularity gives 0.525, and a histogram of the reader's clicks over MIND's category labels 0.598, or 0.617 with the finer subcategory labels. The reference byte-CNN trained on English clicks scores 0.618 from a random start and 0.629 from the distilled weights, so distillation is worth 0.011 here, less than the 0.033 of the June ablation, which drew negatives once and trained for ten epochs. The gap to GloVe NRMS is 0.036 with 165 times fewer parameters, and the distilled encoder is 0.011 above the subcategory histogram, which has no network at all. Feeds outside MIND carry no such editorial labels, which is the case a content encoder is for.

Cross-lingual transfer depends on how the encoder is trained (last column of Table 2). Trained from a random start on English clicks it reads the fourteen translated dev sets at 0.528 on average, close to chance; the models of Table 1 are of this kind. The distilled student alone, frozen under a trained user encoder, scores 0.572 across languages against 0.573 in English, so the distillation does carry the teacher's language alignment (the teacher itself: 0.575 against 0.589). Fine-tuning the distilled encoder on English clicks raises English to 0.629 but leaves the translations at 0.557; showing the same clicks in a random xMIND language instead keeps 0.589 across languages, from 0.538 (Georgian) to 0.609 (Finnish), for 0.609 in English. A trainable linear map on the frozen

Table 2. Reference points and the reference byte-CNN (64-5-384, full precision) on MINDsmall dev. Mean ± standard deviation over 3 seeds; the last column is the mean AUC over the fourteen xMIND translations; "mixed clicks" = each training instance shown in a random xMIND language. Histogram scorers use MIND's editorial labels. Params = news encoder.

Scorer	Params	AUC	MRR	nDCG@10	14-lang. AUC
Popularity	–	0.525	0.255	0.299	–
Category histogram	–	0.598	0.307	0.354	–
Subcategory histogram	–	0.617	0.341	0.384	–
NRMS, reduced (random 256-d table)	7.08 M	0.604±0.010	0.322±0.009	0.364±0.009	–
NRMS, GloVe reference config.	11.29 M	0.664±0.003	0.369±0.002	0.415±0.002	–
Frozen teacher + user encoder	–	0.589±0.000	0.319±0.000	0.360±0.000	0.575±0.000
+ trainable linear map	–	0.618±0.002	0.327±0.001	0.374±0.001	0.590±0.002
Byte-CNN, random start, EN clicks	68 k	0.618±0.002	0.329±0.007	0.371±0.004	0.528±0.002
Byte-CNN, random start, mixed clicks	68 k	0.584±0.005	0.308±0.013	0.348±0.009	0.557±0.001
Byte-CNN, distilled, frozen	68 k	0.573±0.004	0.297±0.002	0.340±0.003	0.572±0.001
Byte-CNN, distilled start, EN clicks	68 k	0.629±0.003	0.338±0.005	0.383±0.003	0.557±0.002
Byte-CNN, distilled start, mixed clicks	68 k	0.609±0.008	0.312±0.004	0.360±0.006	0.589±0.005

teacher reaches 0.590 across languages with the whole teacher behind it; the 68k-parameter byte-CNN comes within 0.001 of that figure.

Two further points qualify the tables. The standard deviation over seeds reaches 0.017 in Table 1, so we read gaps below about 0.02 as unresolved; the June single run of each cell lies within 0.033 of the new mean, the largest difference being the one-bit constrained model (0.521 in June). For the 3% of impressions whose reader has no history, a history-based score ties every candidate (0.500 by construction); popularity gives 0.529 there and the population's category distribution 0.533. By history length, the distilled encoder trained on English clicks scores 0.615 for readers with one to five clicks, 0.633 for six to twenty and 0.638 beyond, against 0.622, 0.664 and 0.688 for GloVe NRMS: the gap widens with history.

Binarising the inner weights the naive way costs the constrained model 0.065 AUC (0.554 over three seeds). Replacing its blocks with full binary convolutions, ReActNet activations and Bi-Real shortcuts, and starting from distilled weights, gives 0.587 ± 0.012 over the same seeds (0.572 in the single June run) at an estimated 194 KB: 0.032 above naive binarisation, 0.018 below the reduced NRMS and 0.042 below the distilled encoder at full precision (Table 2). On the accuracy-versus-footprint view (Fig. 2) eight-bit Micro-NAS (0.614, 204 KB, 3.2 μJ) has the lowest cost at the accuracy of the reduced NRMS, while unconstrained eight-bit leads on accuracy (0.639, 790 KB).

For the boards, the trained constrained encoder (distilled start, English clicks) was exported to ONNX without its byte-table lookup, which the caller performs, and quantised to an 8-bit QDQ model with ONNX Runtime (per-channel 8-bit weights and 8-bit inputs to every convolution and to the output layer, calibrated on 512 training titles; activation functions and averages stay in floating point). The 8-bit file scores 0.622 AUC against 0.628 for the same weights at full precision (0.624 against 0.629 over three seeds). On the STM32H7B3I-DK it runs in 31.2 ms per title, 2.5 times faster than at full precision (77.0 ms), with

53.2 KB of weights, 77.7 KB of flash including the runtime and 49.4 KB of RAM, plus the 64.3 KB byte table; on the NUCLEO-F401RE it takes 450 ms, no faster than full precision there (444 ms). The board's own check of the generated code against the reference model gave zero error. On the phone the 8-bit file of the encoder trained on mixed-language clicks, the one built into our reader app, computes the same vectors as on the laptop (largest difference 4.9×10^{-7} on titles in five scripts) and takes 0.53 ms per title on one core (median over 2,000 calls, from the title's bytes to its vector; 0.48 ms with two threads); the full-precision file takes 0.59 ms. At this size eight bits buy no speed on the phone. The file scores 0.607 AUC in English and 0.592 over the fourteen translations. Encoding in a loop on one core (1,806 titles per second), the unplugged phone draws 2.86 W against 1.15 W with the encoder idle and the screen on, about 0.95 mJ per title. The phone reports only an averaged current and a charge counter in steps of 1 %, which give 0.83 and 1.6 mJ, so we read the cost as roughly one millijoule per title.

In the app the encoder also feeds the ranking of the feed. The app orders articles by a rule it learns on the phone: a weight per topic that grows with every opened or read article, plus terms for drift, freshness and unread items. We ported the rule to Python (the port's scores equal the app's on 600 events and 200 articles) and replayed the MINDsmall clicks through it in time order, scoring each impression before the rule learns from its clicks, with the MIND category as topic and the first appearance of an article in place of its publication time. The content score of a candidate is the dot product of its vector from the app's 8-bit file with the user vector that the user encoder of the same checkpoint pools from the last 50 clicked titles; it enters the rule standardised over the unread candidates of the impression and multiplied by a weight. A candidate already read gets no content term, since a title just read is the closest to the reading history and would otherwise rise to the top of the app's feed. The encoder was trained on the training clicks, so we chose the weight on the earlier half of the dev impressions and report the later half (36,576 impressions; 0.75 % of their candidates had already been read). There the rule alone scores 0.600 AUC and the content score alone 0.594; their sum, with the chosen weight of 4, scores 0.625, 0.025 above the rule (95 % interval 0.022–0.028 over 1,000 resamples of the 28,445 users). With the subcategory as topic the rule scores 0.606 and the sum 0.633 (weight 2). The app ranks with this sum and a weight of 2, the best on the earlier half averaged over both topic definitions; on the phone its content scores for a test history equal the laptop's to within 3.2×10^{-7}.

Table 3 gives the outcome of the board searches. The recipe choices are, within the H7 budget, three convolutions (98, 26 and 94 channels, with one max-pooling) at 2.00 M multiply-accumulates and, within the F401 budget, a strided depthwise and a strided five-wide convolution at 0.65 M. Both lead the best hand-designed shape of their budget in teacher cosine (0.331 against 0.303 and 0.298 against 0.296), but in ranking they only match it: 0.595 against 0.599 AUC within the H7 budget and 0.591 against 0.589 within the F401 budget, differences smaller than the spread over seeds (0.543 against 0.547 and 0.537

Table 3. Encoders under the board budgets. Recipe choice: best of the eight retrained candidates under the short search recipe; final choice: best of the same eight after full distillation. Costs from the search's resource model (MACs per title, eight-bit weights, peak activations). Search cos: validation cosine to the teacher under the search recipe, mean of three seeds (‡: one seed). Teacher cos: English dev titles after distillation on all 769k rows; AUC: MINDsmall dev after English clicks, full precision and as the 8-bit file (INT8). Mean ± standard deviation over 3 seeds. Last two columns: milliseconds per title of the 8-bit file on the STM32H7B3I-DK and the NUCLEO-F401RE. All rows read the bytes through the same fixed table with a padding-mask channel and average over all positions.

Encoder	MACs (M)	Weights (KiB)	Act. (KiB)	Search cos	Teacher cos	AUC	INT8 AUC	H7 (ms)	F401 (ms)
STM32H7B3I-DK budget: 256 KiB weights, 128 KiB activations, 2 M MACs									
μNAS 144, recipe choice	2.00	51.9	15.5	0.331	0.392±0.001	0.595±0.003	0.596±0.003	18.7	252.3
μNAS 140, final choice	1.93	65.8	14.0	0.318	0.398±0.002	0.605±0.010	0.602±0.009	28.4	226.6
Hand-designed 64-2-384	1.66	37.1	16.0	0.303‡	0.371±0.001	0.599±0.005	0.598±0.006	16.6	191.7
NUCLEO-F401RE budget: 128 KiB weights, 48 KiB activations, 1 M MACs									
μNAS 134, recipe choice	0.65	41.5	5.9	0.298	0.349±0.002	0.591±0.003	0.590±0.003	15.1	32.1
μNAS 142, final choice	0.84	60.0	6.5	0.287	0.367±0.001	0.608±0.007	0.605±0.006	12.1	60.6
Hand-designed 32-5-384	1.00	20.2	8.0	0.296‡	0.333±0.019	0.589±0.008	0.589±0.003	21.2	88.7
Outside both MAC budgets									
Hand-designed 64-5-384	3.31	50.1	16.0	0.343	0.459±0.000	0.618±0.006	0.614±0.010	31.5	426.8

against 0.538 across the fourteen translations). Trained on clicks shown in a random language, as in Table 2, the two recipe choices read the translations at 0.562 and 0.556 and English at 0.571 and 0.549. Full distillation reorders the eight finalists of each search: the recipe choices fall to fifth and fourth place, and the final choices are candidate 140 (H7, 1.93 M) and candidate 142 (F401, 0.84 M, with one residual branch). These reach 0.605 and 0.608 AUC, ahead of the hand-designed shapes by 0.006, inside the spread over seeds, and by 0.019. All of them stay below 64-5-384, which fits neither operation budget and scores 0.618 in this form and 0.629 in its original one. Under the search recipe the H7 recipe choice was 0.012 below 64-5-384 in teacher cosine, after full distillation 0.067: a short distillation is a weak predictor of how far a network gets with full training.

On the NUCLEO-F401RE the recipe's F401 choice is the fastest of the seven encoders: 32.1 ms per title, against 88.7 ms for 32-5-384 and 427 ms for 64-5-384. It has a third fewer multiply-accumulates than 32-5-384 but runs 2.8 times faster, at 4.1 cycles per multiply-accumulate against 7.2: in 32-5-384 the depthwise layers take 22 cycles per multiply-accumulate and the first pointwise layer, over the 65 input channels, 8.8. The final F401 choice, larger and more accurate, takes 60.6 ms there, 1.5 times less than 32-5-384, and is the fastest on ST's STM32H7B3I-DK (12.1 ms). Within the H7 budget the hand-designed 64-2-384 is faster on that board (16.6 ms) than both searched choices (18.7 and 28.4 ms). All in-budget encoders keep their ranking accuracy as 8-bit files. Quantising every operator rather than only the convolutions and the output layer kept the accuracy but made no file faster; on the H7 the code generator then fused one convolution with its max-pooling into an integer kernel that took 20.2 ms alone.

On a NUCLEO-F401RE of our own, with ST's validation firmware built from the same code generator, the final F401 choice takes 60.7 ms per title, as on ST's board, and its outputs for 20 MIND and xMIND titles (English, Chinese,

Table 4. Milliseconds per title of the 8-bit files of Table 3 on an STM32H7B3I-DK: ST's benchmark board (Developer Cloud, ST Edge AI Core 4.0.1) and ours (Core 4.0.0, ST's validation firmware) with the code generator's default setting and with time optimisation (`-O time`). Ten random inputs each; on our board the outputs equal those of the ONNX models to within an RMSE of 0.0013.

Encoder	ST's board	Ours, default	Ours, `-O time`
STM32H7B3I-DK budget			
μNAS 144, recipe choice	18.7	44.7	18.8
μNAS 140, final choice	28.4	40.4	19.2
Hand-designed 64-2-384	16.6	32.4	16.5
NUCLEO-F401RE budget			
μNAS 134, recipe choice	15.1	6.7	6.7
μNAS 142, final choice	12.1	12.0	9.9
Hand-designed 32-5-384	21.2	15.9	13.3
Outside both MAC budgets			
Hand-designed 64-5-384	31.5	69.5	31.2

Japanese, Georgian and Finnish) equal those of the ONNX model run on the laptop. With the encoder running back to back, the board draws 0.409 and 0.406 W from the laptop's USB port, against 0.287 W with the MCU in STOP mode. The difference, 23 mA at 5.18 V, is the MCU's current for the encoder: 7.3 mJ per title at the USB input, and about 4.7 mJ at the MCU's 3.3 V supply if a linear regulator feeds it. That is 5.4 nJ per multiply-accumulate, against the 1 pJ of the arithmetic-only estimate behind Table 1: the whole chip runs at 77 mW for the 60.7 ms.

On an STM32H7B3I-DK of our own the same files run faster or slower than on ST's board, depending on where the code generator places the network's buffers (Table 4). The final H7 choice takes 40.4 ms per title with the generator's default setting and 19.2 ms with time optimisation, against 28.4 ms on ST's board; its first convolution alone takes 19.2 or 6.9 ms, with the same weights and the same kernel. Across the seven encoders the times of one file differ by up to 2.4 times. With time optimisation the hand-designed 64-2-384 stays the fastest within the H7 budget (16.5 ms against 18.8 and 19.2 ms), but the fastest encoder on our board is the recipe's F401 choice (6.7 ms, against 15.1 ms on ST's), not the final F401 choice (9.9 ms). For 20 titles the final H7 choice gives the outputs of the ONNX model. With it running back to back, the board draws 1.030 and 1.029 W from USB, against 0.847 W in STOP mode: 0.18 W, or 3.5 mJ per title at 19.2 ms. The board's 3.3 V regulator is not documented, so we give only the USB-side figure.

6 Limitations

The main gap is hardware. We timed the matrix models on the laptop and estimated size, RAM and energy from parameter and operation counts; only the deployed encoders ran on the boards and the phone; the phone's energy comes from its own battery gauge, the boards' from a USB meter on the whole board as the difference between two phases (a probe on the MCU's own supply would

measure it directly). Board times depend on the code generator's version and setting: the same file ran up to 2.4 times faster or slower on our H7 than on ST's. In the matrix the quantisation is simulated on weights only: activations, the byte embedding and the outer layers stay in floating point, so the eight-bit and one-bit rows may overstate accuracy and their sizes are not those of an 8-bit file; for the constrained encoder the 8-bit file costs 0.004 AUC on average over three seeds and for the in-budget encoders of Table 3 at most 0.003. Eight-bit inference runs on STM32 boards through quantised ONNX models and ST Edge AI, as measured above. The one-bit path needs a binary runtime such as CBin-NN [20] or Larq Compute Engine [1], and the Cortex-M7 has no popcount instruction, so we make no speed claim for binary and treat it only as a way to save space. The energy figure ignores SRAM and memory traffic, which can dominate on a Cortex-M. The June constrained search enforced its flash and RAM budgets but not the operation budget. The μNAS searches score a candidate by a short distillation, whose ranking did not survive full training (Sect. 5); the final choice after full distillation of the eight finalists changed both choices, and a longer fitness inside the search would cost more search time. Even after mixed-language training the translations stay 0.020 below English and 0.039 below the English-only encoder. Finally, a byte sequence is about five times longer than the word sequence, trading flash for computation, and all of this is on the small MIND split. The replay through the app's ranking rule has clicks only: MIND records no reading time or scrolling, from which the app also learns, and no publication time, and no reader has used the content-aware ranking yet.

7 Ethics and Conclusion

MIND and xMIND are used under their non-commercial research licences; the work touches no personal data beyond the public logs, and the translations are machine-generated. Replacing the word-embedding table with a byte-level encoder removes the largest item of a news recommender's storage and gives every language the same 256-row vocabulary. With eight-bit weights the constrained encoder reaches 0.614 AUC at an estimated 204 KB, and as an 8-bit file it runs in 31.2 ms per title on an STM32H7B3I-DK; searched under the budget of a NUCLEO-F401RE and chosen after full training, an encoder of 0.84 M multiply-accumulates reaches 0.608 at 60.6 ms per title on that board, 0.019 above the best hand-designed shape of that budget, and draws about 7.3 mJ per title from the USB port of ours; the one-bit model with ReActNet blocks, started from distilled weights, reaches 0.587 at 194 KB; trained on clicks shown in a random language, the distilled byte-CNN reads fourteen languages at 0.589 AUC on average. Added to the ranking rule of our reader app, the content score of its 8-bit encoder raises the AUC of a time-ordered replay of MIND from 0.600 to 0.625. What is still missing: **[TODO: a one-bit runtime on the board]** — that is where we take it next.

References

1. Bannink, T., Bakhtiari, A., Hillier, A., et al.: Larq compute engine: Design, benchmark, and deploy state-of-the-art binarized neural networks. In: Proceedings of MLSys (2021), arXiv:2011.09398
2. Capello, A., Berta, R., Ballout, H., Fresta, M., Soltanmuradov, V., Bellotti, F.: Enhancing μNAS for 1D CNNs on microcontrollers. In: Proceedings of SIE 2024, Lecture Notes in Electrical Engineering, vol. 1263, pp. 481–486. Springer (2025). https://doi.org/10.1007/978-3-031-71518-1_59
3. Clark, J.H., Garrette, D., Turc, I., Wieting, J.: CANINE: Pre-training an efficient tokenization-free encoder for language representation. In: Transactions of the ACL (2022)
4. Edizel, B., Mantrach, A., Bai, X.: Deep character-level click-through rate prediction for sponsored search. In: Proceedings of SIGIR. pp. 305–314 (2017). https://doi.org/10.1145/3077136.3080811
5. Gillick, D., Brunk, C., Vinyals, O., Subramanya, A.: Multilingual language processing from bytes. arXiv:1512.00103 (2016)
6. Horowitz, M.: Computing's energy problem (and what we can do about it). In: IEEE ISSCC. pp. 10–14 (2014)
7. Iana, A., Glavaš, G., Paulheim, H.: MIND your language: A multilingual dataset for cross-lingual news recommendation. In: Proceedings of SIGIR (2024). https://doi.org/10.1145/3626772.3657867
8. Kaliamoorthi, P., Ravi, S., Kozareva, Z.: PRADO: Projection attention networks for document classification on-device. In: Proceedings of EMNLP-IJCNLP (2019). https://doi.org/10.18653/v1/D19-1506
9. Kusupati, A., Bhatt, G., Rege, A., Wallingford, M., Sinha, A., Ramanujan, V., Howard-Snyder, W., Chen, K., Kakade, S., Jain, P., Farhadi, A.: Matryoshka representation learning. In: Proceedings of NeurIPS (2022)
10. Liberis, E., Dudziak, Ł., Lane, N.D.: μNAS: Constrained neural architecture search for microcontrollers. Proceedings of EuroMLSys (2021), arXiv:2010.14246
11. Lin, J., Chen, W.M., Lin, Y., Cohn, J., Gan, C., Han, S.: MCUNet: Tiny deep learning on iot devices. In: Proceedings of NeurIPS (2020)
12. Lin, Z., Wang, Y., Zhang, J., Chu, X., Ling, H.: NAS-BNN: Neural architecture search for binary neural networks. Pattern Recognition **159**, 111086 (2025). https://doi.org/10.1016/j.patcog.2024.111086
13. Liu, Z., Shen, Z., Savvides, M., Cheng, K.T.: ReActNet: Towards precise binary neural network with generalized activation functions. In: Proceedings of ECCV (2020)
14. Liu, Z., Wu, B., Luo, W., Yang, X., Liu, W., Cheng, K.T.: Bi-real net: Enhancing the performance of 1-bit cnns with improved representational capability and advanced training algorithm. In: Proceedings of ECCV (2018)
15. Mao, Z., Li, J., Wang, H., Zeng, X., Wong, K.F.: DIGAT: Modeling news recommendation with dual-graph interaction. In: Findings of EMNLP (2022), arXiv:2210.05196
16. NLLB Team: No language left behind: Scaling human-centered machine translation (2022)
17. Pighetti, A., Bellotti, F., Mohammady, S., Lazzaroni, L., Ballout, H., Berta, R.: Exploring neural architecture search (NAS) for binary neural networks (BNNs) in the new WakeVision. In: Applications in Electronics Pervading Industry, Environment and Society (ApplePies), Lecture Notes in Electrical Engineering, vol. 1553, pp. 85–90 (2026). https://doi.org/10.1007/978-3-032-17174-0_12

18. Rastegari, M., Ordonez, V., Redmon, J., Farhadi, A.: XNOR-Net: Imagenet classification using binary convolutional neural networks. In: Proceedings of ECCV (2016)
19. Reimers, N., Gurevych, I.: Making monolingual sentence embeddings multilingual using knowledge distillation. In: Proceedings of EMNLP (2020)
20. Sakr, F., Berta, R., Doyle, J., Capello, A., Dabbous, A., Lazzaroni, L., Bellotti, F.: CBin-NN: An inference engine for binarized neural networks. Electronics **13**(9), 1624 (2024). https://doi.org/10.3390/electronics13091624
21. Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., Zhou, M.: MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In: Proceedings of NeurIPS (2020)
22. Wu, C., Wu, F., An, M., Huang, J., Huang, Y., Xie, X.: Neural news recommendation with attentive multi-view learning. In: Proceedings of IJCAI (2019)
23. Wu, C., Wu, F., Ge, S., Qi, T., Huang, Y., Xie, X.: Neural news recommendation with multi-head self-attention. In: Proceedings of EMNLP-IJCNLP. pp. 6389–6394 (2019)
24. Wu, C., Wu, F., Huang, Y., Xie, X.: Personalized news recommendation: Methods and challenges. ACM Transactions on Information Systems (2023), arXiv:2106.08934
25. Wu, F., et al.: MIND: A large-scale dataset for news recommendation. In: Proceedings of ACL. pp. 3597–3606 (2020)
26. Zhang, X., Zhao, J., LeCun, Y.: Character-level convolutional networks for text classification. In: Advances in Neural Information Processing Systems 28 (2015), arXiv:1509.01626